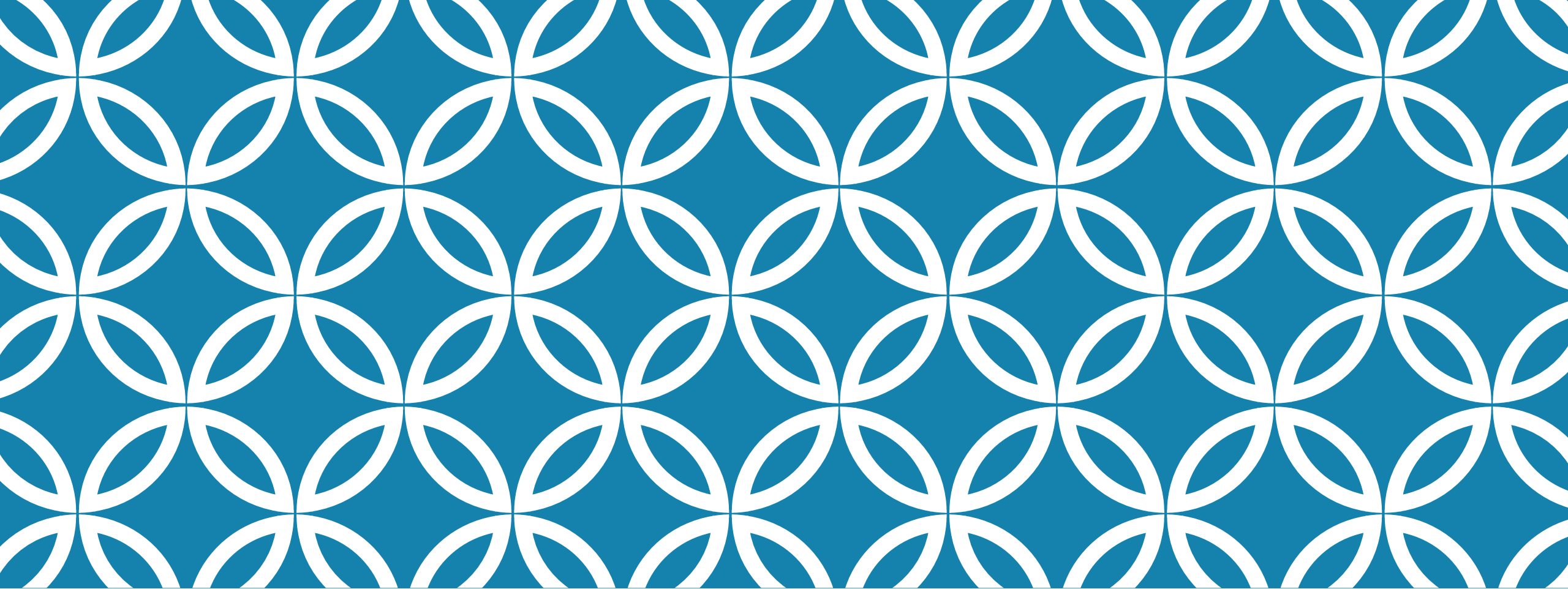




STATIC PROGRAM ANALYSIS

Lecture 5 – Reaching Definitions,
Data-flow framework.

SE QUALITY PROBLEM

All assignments to variables must be used somewhere

- Identifies assignment locations
- Identify usage locations
- Make sure that after executing an assignment location a program executes a usage location with no-reassignments in between.

Need to know the order of

- Somewhat challenging to do it over AST – need to remember way too much information
- Easier way is to propagate this information over a control flow graph CFG
- Reaching definition – a classic data-flow analysis
- WALA builds a CFG for you

SE QUALITY/OPTIMIZATION PROBLEM

All assignments to variables must be used somewhere

- Identifies assignment locations
- Identify usage locations
- Make sure that after executing an assignment location a program executes a usage location with no-reassignments in between.

```
1: int compute (int y){  
2: int x = 2;  
3: if(x < y){  
4:  y = 23;  
   } else {  
5:  x = 17}  
6: return x;}
```

Definitions of x: 2 and 5
Usage of x: 3 and 6

```
1: int compute (int y){  
2: int x = 2;  
3: if(x < y){  
4:  y = 23;  
   } else {  
5:  x = 17}  
6: return x;}
```

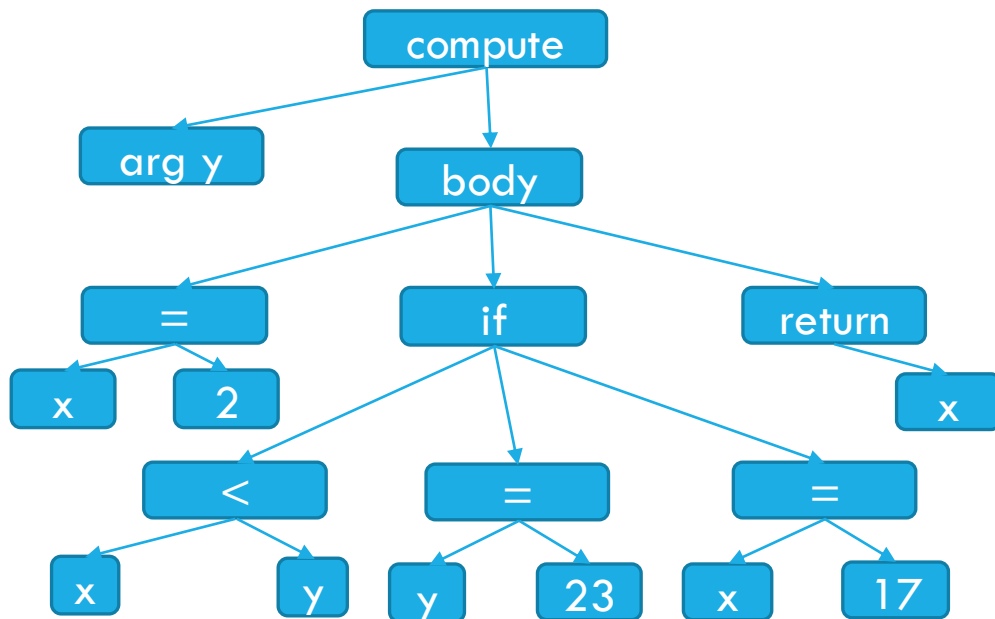
Def 2 → Use 3
Def 5 → Use 6

What about y?

CHOOSING RIGHT ABSTRACTION

Abstract Syntax Tree (AST) flow-insensitive analysis

Difficult to do with just visitors, need to remember traversing order, which simple node belongs where.



```
1: int compute (int y){
```

```
2: int x = 2;
```

```
3: if(x < y){
```

```
4: y = 23;
```

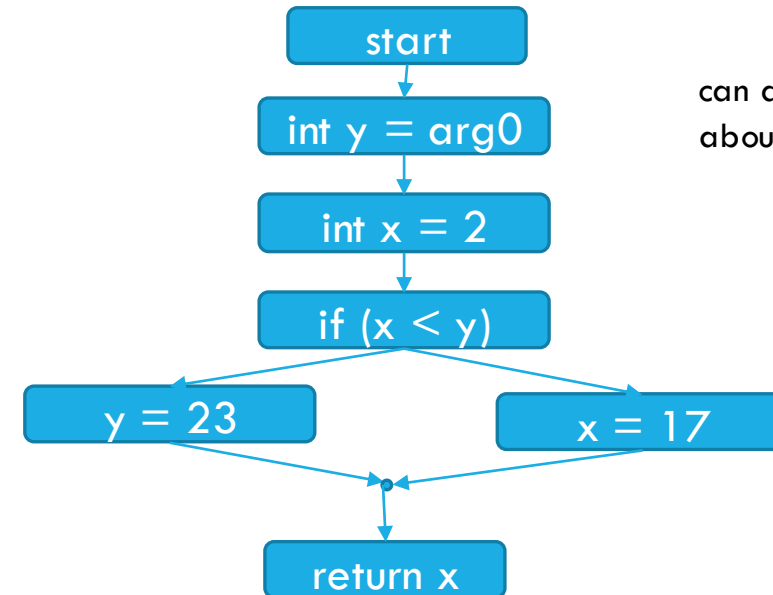
```
    } else {
```

```
5: x = 17}
```

```
6: return x;}
```

Control Flow Graph (CFG) – flow sensitive analysis

Keeps track of partial order on statements and 3 address code, e.g., $x = y + 2 + z \Rightarrow t0 = 2 + z; x = y + t0$.



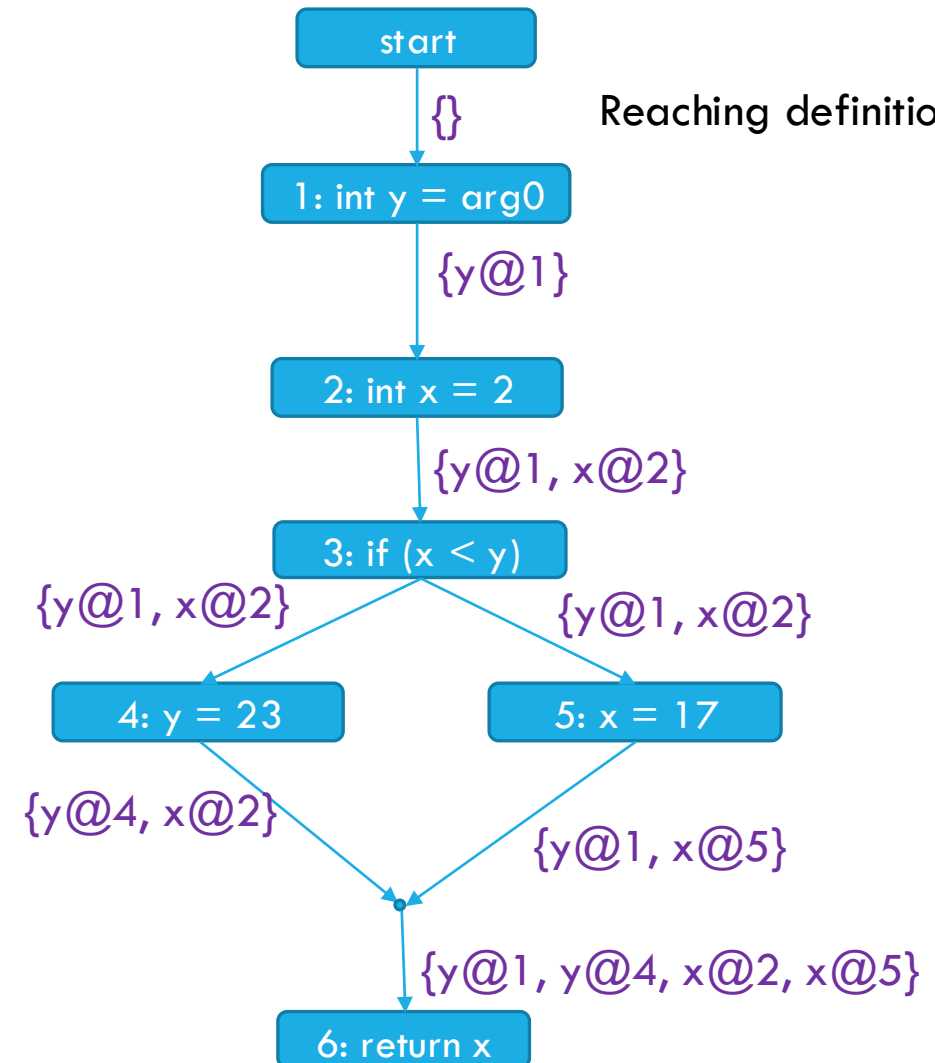
can query each statement about its lhs, rhs, expressions

WHAT INFORMATION TO COMPUTE

Set of definitions that could be reached at each statement $\{y@1, x@2\}$

Post processing of analysis results

- Analyze each statement and determine usage
- assignments: rhs has y and the incoming definition set is $\{y@1, x@2\}$ then mark $y@1$ as being used
- conditional: analyze expression e.g., $x < y$, and if incoming definition set is $\{y@1, x@2\}$ then mark $y@1$ and $x@2$ as being used



HOW TO COMPUTE OVER CFG

Start with $\{\}$

An assignment statement

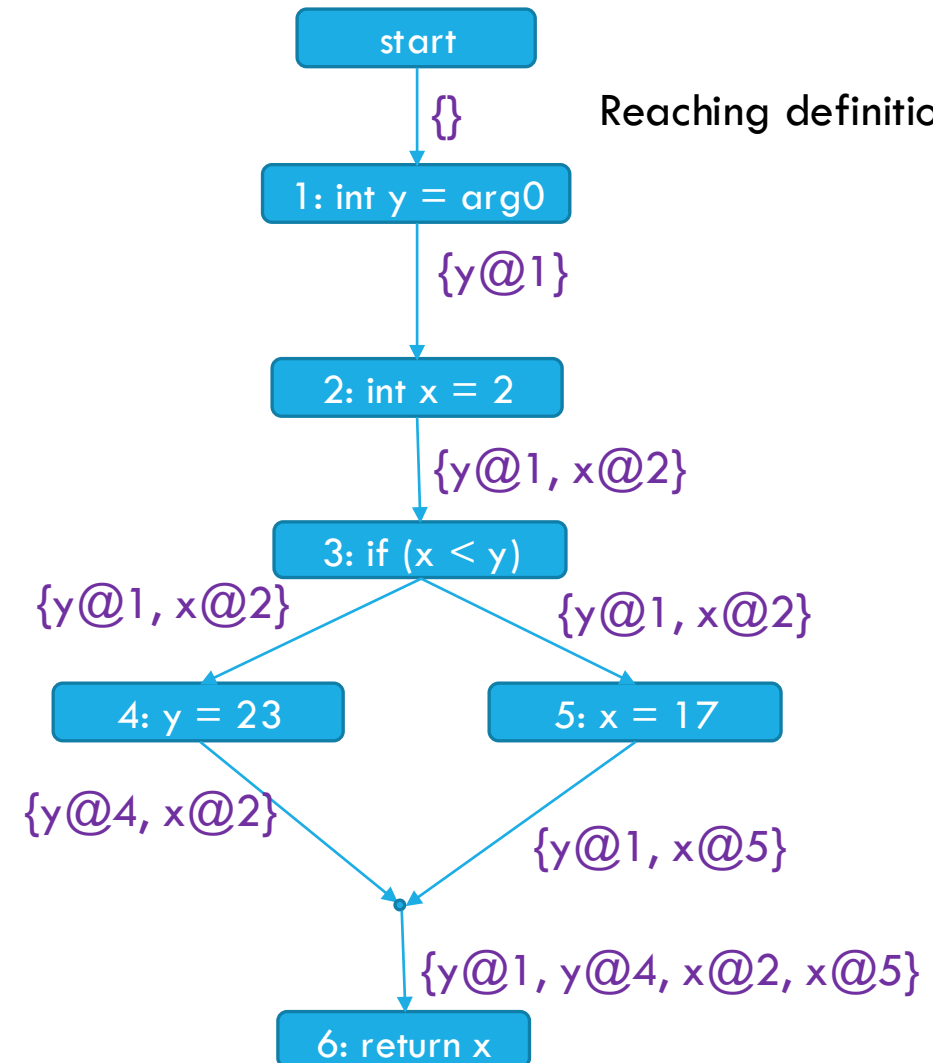
- find name of variable on lhs
- remove all its definition from the incoming set
- add the current definition

Example

- incoming $\{x@1, y@2, y@5\}$
- statement 7: $y = y - 2;$
- removing all $y@$: $\{x@1\}$
- adding stmt7 definition: $\{x@1, y@7\}$

For each node we keep track of incoming sets and outgoing sets

If a node has two incoming edges then we union sets (as in stmt 6).



IMPLEMENTATION IN AN ANALYZER

Every static analyzer provides with a CFG

Properties

- Start node
- Parents children
- Queriable nodes
 - a conditional statement
 - an assignment statement

What order to execute: BFS or DFS



ALGORITHM COMPONENTS

Traversing

Data-structures

Merging of information flows



DEVELOPING PSEUDO-CODE

Let's complete it together

NEXT TIME

Implementing this pseudo-cod in WALA

Install WALA in Eclipse

- I will do the same on my new machine
- I will do basic development in our repository